

User Question Processing

Code Reference - Telegram to LangGraph to deterministic search and reply

MASTER FLOW

Telegram user -> **Bot API / polling** -> `handle_message()` -> `_process_with_typing()` -> `_process_user_message()` -> **thread ID** -> `_invoke_graph_async()` -> `graph.invoke()` -> **restore conversation state** -> `prepare_conversation_request()` -> `understand_request()` -> **routing** -> **search or clarification** -> **build final_answer** -> **checkpoint** -> **result** -> `message.reply_text()` -> **Telegram user**.

FOUR LAYERS TO REMEMBER

- **Telegram layer** - receives updates and sends replies. File: `agent/telegram_bot.py`.
- **Conversation + graph layer** - restores memory, understands follow-ups and controls routing. Files: `graph.py` and `conversation.py`.
- **Request understanding layer** - converts free text into structured search filters. File: `request_parser.py`.
- **Search + response layer** - deterministic activity filtering and formatting. File: `tools.py` plus response helpers in `graph.py`.

AI USAGE MAP

- `classify_conversation_message()` may use GPT-4o-mini to classify how a new message relates to prior context.
- `parse_user_request()` uses GPT-4o-mini to produce a structured `ParsedRequest`.
- `reinterpret_unclear_request()` is a second interpretation attempt only when needed.
- The actual activity search is deterministic. GPT does not choose activities from the database.

TWO IMPORTANT DISTINCTIONS

- `graph.invoke()` does not directly mean "call GPT". It runs the complete LangGraph flow with memory and routing.
- Conversation memory is stored through `PostgresSaver`. Telegram `context.user_data` is only temporary UI state such as pagination.

END RESULT

A natural-language Telegram message becomes a structured request, is routed safely, searched deterministically against normalized activities already loaded in memory, then returned as `final_answer` and sent back through the Telegram Bot API.

1. Telegram Interface and Entry Point

1.1 Backend and Bot API

- Telegram does not run the Python code. The Python backend is a separate process that communicates through the Telegram Bot API.
- File: `agent/telegram_bot.py`.
- Authentication uses `TELEGRAM_BOT_TOKEN` from the environment, not a hard-coded token.

1.2 Polling and handler registration

- `main()` starts the application and uses `application.run_polling()`.
- `build_application()` registers handlers before a specific user message arrives.
- A normal text message that is not a command is routed to `handle_message()`.
- If the backend process stops, polling stops and new messages are not processed.

```
application.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, handle_message))
application.run_polling(...)
```

1.3 Read the user text

- `handle_message()` receives the Telegram Update.
- It reads `update.effective_message`, then `message.text`.
- At this point `user_message` is simply the string written by the user.

```
message = update.effective_message
user_message = (message.text or "").strip()
```

1.4 Typing UX and main Telegram processing function

- `handle_message()` calls `_process_with_typing()`.
- `_process_with_typing()` only manages the Typing indicator while the real processing continues in `_process_user_message()`.
- `_process_user_message()` handles simple Telegram-side cases first, such as greetings, thanks and pagination, before invoking the Agent.

1.5 Build a stable conversation thread ID

- `_get_conversation_thread_id()` combines the Telegram `chat_id` and `user_id`.
- Messages from the same user in the same chat therefore reuse the same `thread_id`.
- That stable ID is what allows LangGraph to associate later messages with the same saved conversation state.

```
return f"telegram:{chat_id}:{user_id}"
```

2. Graph Entry, Persistent Memory and AgentState

2.1 Telegram -> Graph bridge

- `_process_user_message()` calls `_invoke_graph_async()`.
 - `_invoke_graph_async()` passes the user text as Graph input and passes the stable `thread_id` inside the LangGraph config.
 - The actual graph call is `graph.invoke(...)`.
- ```
config = {"configurable": {"thread_id": thread_id}}
graph.invoke({"user_message": user_message}, config)
```

### 2.2 Persistent conversation memory

- File: `agent/graph.py`.
  - `DATABASE_URL` is used to connect to PostgreSQL.
  - `ConnectionPool` keeps reusable database connections so a new connection is not opened for every checkpoint operation.
  - `PostgresSaver(database_pool)` is the LangGraph checkpointer used for persistent conversation state.
- ```
database_pool = _build_database_pool()
checkpointer = PostgresSaver(database_pool)
checkpointer.setup()
```

2.3 What LangGraph does with `thread_id`

- When `graph.invoke()` runs with a thread ID, LangGraph restores the relevant previous checkpoint for that thread.
- During the graph run, new checkpoints are written automatically through the checkpointer.
- `prepare_conversation_request()` does not manually `SELECT` the memory table. The restore/save behavior belongs to LangGraph + `PostgresSaver`.

2.4 AgentState

- File: `agent/state.py`.
- `AgentState` is the shared state object passed between graph nodes.
- Important fields include `user_message`, `intent`, `interpretation_confident`, `category`, `age`, `day`, `start_after`, `start_before`, `location`, `center_name`, `branch`, `instructor`, `clarification` fields, `tool_results` and `final_answer`.
- Each node reads what it needs and returns updates to the same shared state.

2.5 Resetting persistent conversation memory

- `delete_conversation_thread(thread_id)` calls `checkpointer.delete_thread(...)`.
- This is the persistent-memory reset used when starting a truly new conversation state.

3. Conversation Context and Follow-up Handling

3.1 First graph node: `prepare_conversation_request()`

- File: `agent/graph.py`.
- This node decides how the new message should be interpreted relative to the previous conversation state.
- Example: a prior search for Pilates on Tuesday evening followed by "and on Wednesday?" should preserve the Pilates/evening context and change the day.

3.2 Deterministic gate before GPT classification

- `has_meaningful_search_filters(previous_state)` checks whether a useful previous search context exists.
- `should_check_conversation_context(user_message, previous_state)` checks whether the new message looks like it may depend on that prior context.
- Therefore not every message is automatically sent for conversation-context classification.

3.3 Optional conversation classification

- If context classification is needed, `prepare_conversation_request()` calls `classify_conversation_message()` in `agent/conversation.py`.
- `state_to_context_text()` converts saved filters into clear context text for the model.
- The model is GPT-4o-mini with temperature 0 and structured output `ConversationDecision`.

3.4 `ConversationDecision`

- `action` describes how to treat the message, for example `follow_up`, `new_query` or `use_known_filters`.
- `query_fragment` contains the new or changed part of the user message.
- `clear_fields` lists filters the user asked to remove.

3.5 Merge a follow-up

- For `follow_up`, `merge_follow_up()` keeps the previous filters, clears requested fields, parses the new fragment and overwrites only fields that changed.
- The function rebuilds a complete query that represents the full current intent before the next parsing stage.
- For `new_query`, the new request is not merged with the previous search filters.

4. Request Understanding, Routing and Clarification

4.1 understand_request()

- Back in `agent/graph.py`, the next node is `understand_request()`.
- It calls `parse_user_request()` from `agent/request_parser.py`.
- The role here is different from conversation classification: this stage asks what the user is actually searching for.

4.2 ParsedRequest structured output

- `parse_user_request()` uses GPT-4o-mini with structured output `ParsedRequest`.
- Important fields include `intent`, `interpretation_confident`, `age`, `category`, `target_audience`, `day`, `start_after`, `start_before`, `location`, `center_name`, `branch` and `instructor`.
- GPT only structures the request. It does not query Supabase and does not select matching activities.

4.3 Deterministic post-processing

- After the model output, `request_parser.py` applies deterministic rules for relative days, time ranges, morning/evening expressions and intent confidence.
- The goal is to normalize values and reduce unsupported or invented fields before they enter the Graph state.

4.4 route_after_understanding()

- This is deterministic routing logic in `graph.py`.
- If `intent == "activity"` and `interpretation_confident == True`, the flow goes to `activity_node()`.
- Otherwise the flow goes to `fallback_node()`, unless the state already requires clarification.

4.5 fallback_node() and reinterpret_unclear_request()

- `fallback_node()` calls `reinterpret_unclear_request(user_message)` from `request_parser.py`.
- This is a second attempt to understand the request itself using a prompt designed for unclear or typo-heavy input.
- It is not a request to repair the first `ParsedRequest`.
- `route_after_fallback()` then decides between `activity_node()` and `clarification_node()`.

4.6 clarification_node()

- `clarification_node()` does not search activities and does not invent missing filters.
- It builds a question from information already known in state, sets `waiting_for_clarification=True`, stores the question in `clarification_question`, and also places it in `final_answer`.
- A missing optional field does not automatically require clarification. Clarification is used when the search intent itself cannot be understood confidently.

5. Clarification Loop and Activity Search

5.1 What happens when the user answers a clarification question

- Telegram treats the answer as a normal new message: `handle_message()` -> `_process_user_message()` -> same `thread_id` -> `graph.invoke()`.
- Because the thread ID is unchanged, LangGraph restores the prior state where `waiting_for_clarification=True`.
- `prepare_conversation_request()` can then classify the new message as a clarification response/follow-up and merge it with filters already known.
- The graph starts a new invocation; it does not literally resume execution from the previous node.

5.2 `activity_node()`: AI no longer chooses activities

- Once the request is clear, the flow reaches `activity_node()` in `graph.py`.
- `activity_node()` calls `search_activities()` from `agent/tools.py` with filters already stored in state.
- The search itself is deterministic Python logic.

5.3 Where the activity list comes from

- The Agent does not reopen DOCX, Excel or Publuu sources during each user question.
- `tools.py` defines `reload_data()`, which calls `get_all_activities()` from `database/activities_repository.py`.
- `get_all_activities()` reads the normalized rows from the Supabase PostgreSQL `activities` table.
- The resulting list is stored in Python memory and used by the search layer.

```
lecturer_activities = get_all_activities()
activities = lecturer_activities
```

5.4 When `reload_data()` runs

- `reload_data()` is called when `tools.py` loads.
- The background source synchronization also passes `refresh_data=reload_data`, so the in-memory list is refreshed after source synchronization.
- A new database query is not performed for every user question.

5.5 Deterministic search filters

- `search_activities()` iterates over the in-memory activity list.
- It can filter by category/activity name, age, day, start-time range, location, target audience, center, branch and instructor.
- Cancelled activities are excluded by default.
- For age, explicit matches are kept and unknown age information can remain as lower-certainty candidates; definite age mismatches are removed.

6. Response Building and Return to Telegram

6.1 Format activities

- File: `agent/tools.py`.
- `format_activity_hebrew()` converts one activity dictionary into readable Hebrew output.
- This formatting step is deterministic; it is not a final GPT-generated answer.

6.2 Build the limited first response

- Inside `graph.py`, `activity_node()` uses helper logic such as `_build_limited_answer()` to create the visible response.
- The Graph stores both the full search information in `tool_results` and the user-facing text in `final_answer`.

6.3 Checkpoint and return value

- As the Graph completes, `LangGraph` saves the updated state through `PostgresSaver`.
- The return value of `graph.invoke()` is passed back to `telegram_bot.py` as `result`.
- `_process_user_message()` reads `final_answer`, `tool_results` and clarification/result flags from that returned state.

6.4 Telegram UI state and pagination

- Telegram-side `context.user_data` is used for temporary interface state such as `waiting_for_more`, `pagination_results` and `pagination_offset`.
- This is separate from persistent Agent conversation memory stored through `PostgresSaver`.
- Buttons can be converted into natural follow-up messages such as "and in the evening?" and sent through the same processing path.

6.5 Send the final answer

- The final network-facing send happens in `agent/telegram_bot.py`.
- `message.reply_text(final_answer, ...)` sends the text through the Telegram library and Bot API back to the user chat.
- The `LangGraph` END node itself does not send anything to Telegram; the Telegram layer sends the returned `final_answer`.

```
result = await _invoke_graph_async(...)
final_answer = result.get("final_answer")
await message.reply_text(final_answer, reply_markup=response_keyboard)
```

7. Responsibility and Runtime Cheat Sheet

7.1 File responsibility map

- `telegram_bot.py` - Telegram connection, polling, handlers, thread ID, pagination/UI state and sending replies.
- `graph.py` - flow orchestration, memory checkpointer, routing, clarification, activity-node response assembly.
- `conversation.py` - relationship between a new message and previous search context; follow-up merge logic.
- `request_parser.py` - converts current/merged free text into structured search filters.
- `state.py` - shared `AgentState` definition.
- `tools.py` - in-memory activity data, deterministic filtering and activity formatting.
- `activities_repository.py` - reads normalized activities from Supabase PostgreSQL.

7.2 AI vs deterministic logic

- **AI:** conversation relation classification when needed, request parsing, and optional fallback reinterpretation.
- **Deterministic:** routing decisions, actual activity filtering, validation of many normalized values, response formatting, pagination and final Telegram send.
- This separation prevents the model from directly choosing arbitrary database results.

7.3 Memory vs search data

- **Conversation memory:** LangGraph checkpoints stored in PostgreSQL through `PostgresSaver`, keyed by `thread_id`.
- **Search data:** normalized activities read from Supabase PostgreSQL and held in the Python `activities` list.
- **Telegram UI state:** temporary pagination/button state in `context.user_data`.
- These are three separate responsibilities and should not be mixed together.

7.4 Runtime path for one normal clear search

```
handle_message() -> _process_user_message() -> thread ID -> graph.invoke() -> restore state ->
prepare_conversation_request() -> understand_request() -> parse_user_request() ->
route_after_understanding() -> activity_node() -> search_activities() -> _build_limited_answer()
-> checkpoint -> message.reply_text().
```

7.5 Runtime path for an unclear search

```
understand_request() -> route_after_understanding() -> fallback_node() ->
reinterpret_unclear_request() -> route_after_fallback() -> either activity_node() or
clarification_node().
```

8. Show Me in the Code

נושא	File	Function / Line
חיבור הבוט לטלגרם	agent/telegram_bot.py	_get_bot_token() / TELEGRAM_BOT_TOKEN
הפעלת פולינג	agent/telegram_bot.py	main() / application.run_polling()
רישום מטפלי הודעות	agent/telegram_bot.py	build_application()
קבלת הודעת טקסט	agent/telegram_bot.py	handle_message()
עיבוד הודעת המשתמש	agent/telegram_bot.py	_process_user_message()
בניית מזהה שיחה	agent/telegram_bot.py	_get_conversation_thread_id()
העברת ההודעה לגרף	agent/telegram_bot.py	_invoke_graph_async() / graph.invoke()
בניית חיבור למסד הזיכרון	agent/graph.py	_build_database_pool()
שמירת זיכרון השיחה	agent/graph.py	PostgresSaver(database_pool)
מבנה מצב הסוכן	agent/state.py	AgentState
הכנת בקשה לפי ההקשר	agent/graph.py	prepare_conversation_request()
בדיקת מסננים קודמים	agent/conversation.py	has_meaningful_search_filters()
בדיקת צורך בהקשר שיחה	agent/conversation.py	should_check_conversation_context()
סיווג הודעת המשך או חיפוש חדש	agent/conversation.py	classify_conversation_message()
מיזוג הודעת המשך	agent/conversation.py	merge_follow_up()
ניתוח בקשת המשתמש	agent/request_parser.py	parse_user_request()
ניתוב אחרי הניתוח	agent/graph.py	route_after_understanding()
ניסיון פירוש נוסף	agent/graph.py / agent/request_parser.py	fallback_node() / reinterpret_unclear_request()
ניתוב אחרי ניסיון נוסף	agent/graph.py	route_after_fallback()
בניית שאלת הבהרה	agent/graph.py	clarification_node()
חיפוש הפעילויות	agent/tools.py	search_activities()
טעינת הפעילויות מהמסד	agent/tools.py / database/activities_repository.py	reload_data() / get_all_activities()
עיצוב פעילות לתשובה	agent/tools.py	format_activity_hebrew()
בניית התשובה	agent/graph.py	activity_node() / _build_limited_answer()
שלחת התשובה למשתמש	agent/telegram_bot.py	message.reply_text()

ONE SENTENCE TO REMEMBER

Telegram receives the message and passes it to LangGraph; LangGraph restores conversation state, understands the request and routes it; the search itself filters already-loaded activities deterministically; then the Graph returns `final_answer` and `telegram_bot.py` sends it back with `message.reply_text()`.